Programming II

Building, Running and Deployment in Java

Michele Pasqua, PhD

UNIVERSITÀ
di VERONA
Dipartimento
di INFORMATICA

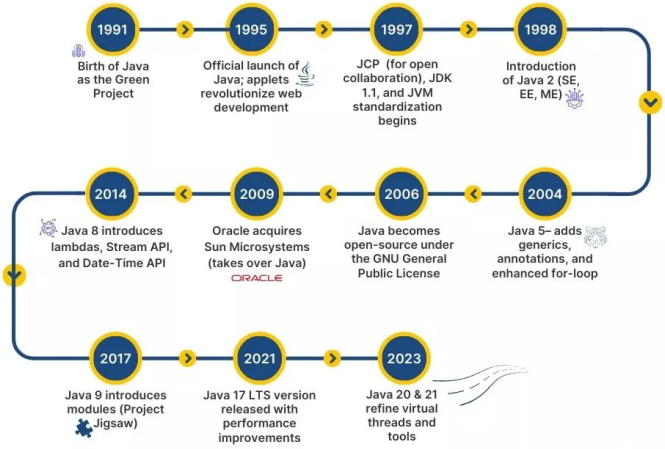
A.Y. 2025/2026

The Java language

Brief Java history

Java timeline

James Gosling



Java features

Java is a robust language, i.e., less error prone

- Strongly typed // *compile-time checks*
- No explicit pointer manipulation // *less memory errors*
- Runtime checks // *no array overflow*
- Automated garbage collection // *less memory leaks*
- Errors are checked by programmers with exceptions

Java is a dynamic language // *not in the sense of PL theory*

- Runtime loading and linking
- Dynamic array size

Java features (cont'd)

Java is easy to learn and use

- Syntax close to C/C++
- Quasi-pure object-oriented language
- Easy building and deployment

Java is portable

- Platform independent
- Translated to intermediate language and then interpreted

Java is not slow as they say (many optimizations, e.g., JIT)

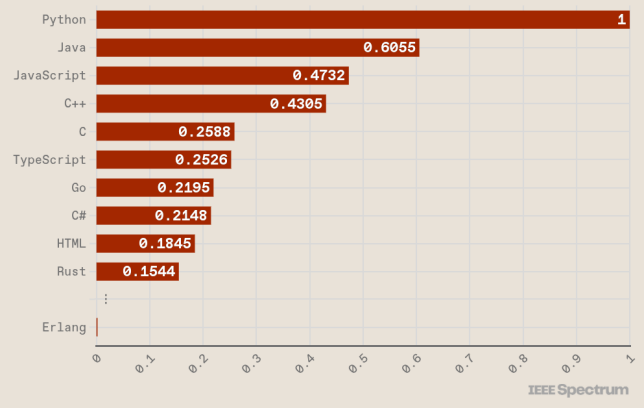
Java is not JavaScript

"Java is to JavaScript what Car is to Carpet." Christian Heilmann

JavaScript name is part of a marketing strategy from Netscape

Java popularity

Top programming languages 2024



Portability: the bytecode

Java is an interpreted language

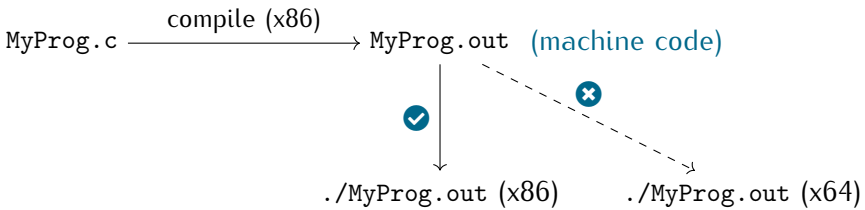
- The compiler does not translate directly to machine code
- Source code is translated to an intermediate language called bytecode
- The bytecode is interpreted on the target machine by using the JVM

Write once, run everywhere

- Instead of distributing executables (for any target machine)
- Distribute an interpreter (for any target machine)
- Then, the same bytecode can be seamlessly run on all machines

Compiled vs interpreted

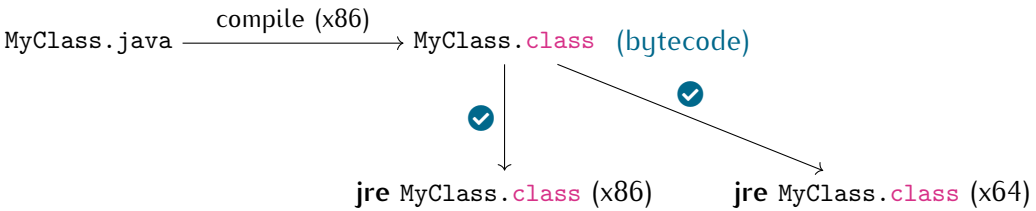
Deployment with a compiled language (like C/C++)



- Cross-platform support is a programmer burden

Compiled vs interpreted (cont'd)

Deployment with an interpreted language (like Java)



- Cross-platform support is a language developer burden

Java ecosystem

... something you can use to write Java programs

Java Development Kit – **jdk**

- Java debugger (`jdb`)
- Java compiler (`javac`)
- Java disassembler (`javap`)
- Java documentation (`javadoc`)

Java Runtime Environment – **jre**

- Class libraries (standard APIs)
- Just-In-Time (JIT) compiler
- Java Virtual Machine (JVM)
- Java application launcher (`java`)

Java Program Deployment

First Java program

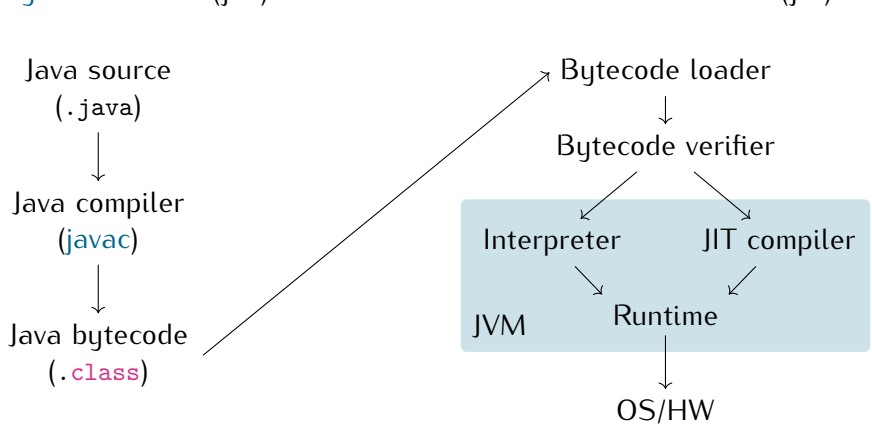
A Java program consists in a class, to be stored into a file with the same name

```
// My first Java program -- To be saved into a file named MyClass.java
public class MyClass {

  // entrypoint of the program
  public static void main(String[] args) {
    System.out.println("Hello World!");
  }

}
```

To generate the bytecode: `javac MyClass.java`

- This will create a file named `MyClass.class`

Building and running



Dynamic class loading

The JVM class loading is based on the classpath

- List of locations where classes can be loaded

When a class `Clazz` is needed

- Look for file `Clazz.class` in the first location of the classpath
- If not present, look in the next classpath location

Dynamic class loading (cont'd)

```
// My first Java program -- To be saved into a file named MyClass.java
public class MyClass {

  // entrypoint of the program
  public static void main(String[] args) {
    System.out.println("Hello World!");
  }

}
```

To run the bytecode: `java -cp . MyClass`

- This tells the JVM to look for classes in the current directory (i.e., .)
- `System.out` is not in ., so the JVM will look into the system classpath

Deployment

Java programs are usually packed and deployed in `jar` files

- Compressed archives containing the bytecode
- Contain additional meta-information
- The JVM can load classes from a `jar` file

To create a `jar` file: `jar cvf MyJar.jar MyClass.class`

- One can pack multiple classes, e.g., `*.class`

To run a class from a `jar` file: `java -cp MyJar.jar MyClass`

Deployment (cont'd)

If a main class is defined in a `jar` file, one can simply type: `java -jar MyClass.jar`

To define a main class, a custom manifest file must be added

```
jar cvfm MyClass.jar manifest.txt MyClass.class
```

where `manifest.txt` contains the line `Main-Class: MyClass`

The manifest file is stored into the `jar` file as: `META-INF/MANIFEST.MF`

Java Development Environment

Text editor

First, one needs to install the `jdk` *// usually, the `jre` comes with the `jdk`*

`apt install default-jdk` or `apt install openjdk-17-jdk`

Then, one just needs a text editor to program *// like `gedit`, `vim` or `emacs`*

Workflow

- Save a program in one or more `.java` files
- Compile the Java files with `javac` to obtain the bytecode (`.class` files)
- Optionally pack the bytecode into a `jar` archive
- Run the JVM on the bytecode with `java`

Integrated Development Environment

An IDE is a toolchain to support programmers

Available IDEs for Java

- IntelliJ IDEA `https://www.jetbrains.com/idea/download`
- Eclipse `https://www.eclipse.org/downloads`
- NetBeans `https://netbeans.apache.org/front/main/download`
- Visual Studio Code `https://code.visualstudio.com/docs/languages/java`

Integrated Development Environment (cont'd)

Advantages of an IDE

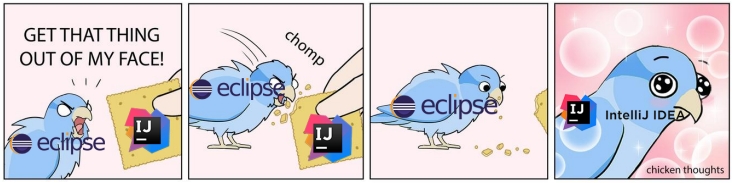
- Provide syntax highlighting
- Provide live code linting // *type checking, code smells, etc.*
- Add pretty printing support
- Simplify code running and test // *integrated debugger*
- Provide graphical project organization and navigation
- Integrate version control tools // *git*
- Integrate build tools // *for Java: `maven`, `gradle` or `ant`*
- Provide automated code generation, do not use when learning

Which IDE?


Does she code in Java?
Yeah
Of course honey! It's so intuitive and easy to use!
Eclipse is a great IDE isn't it?
your foster parents are dead

Which IDE? (cont'd)

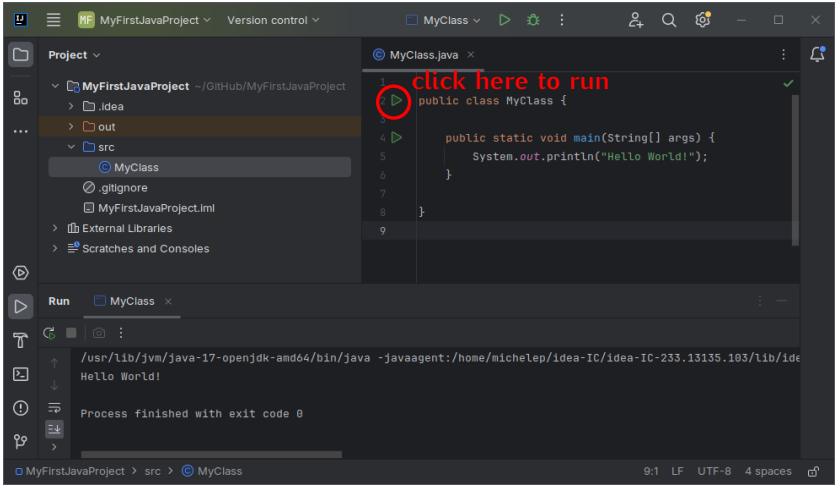
You can choose the IDE you prefer, in the lab we will use IntelliJ IDEA



It is a proprietary software, but

- The Community Edition is free to use
- The Ultimate Edition is still free for university students

First Java project



Q + A